

OpenMP Tasks

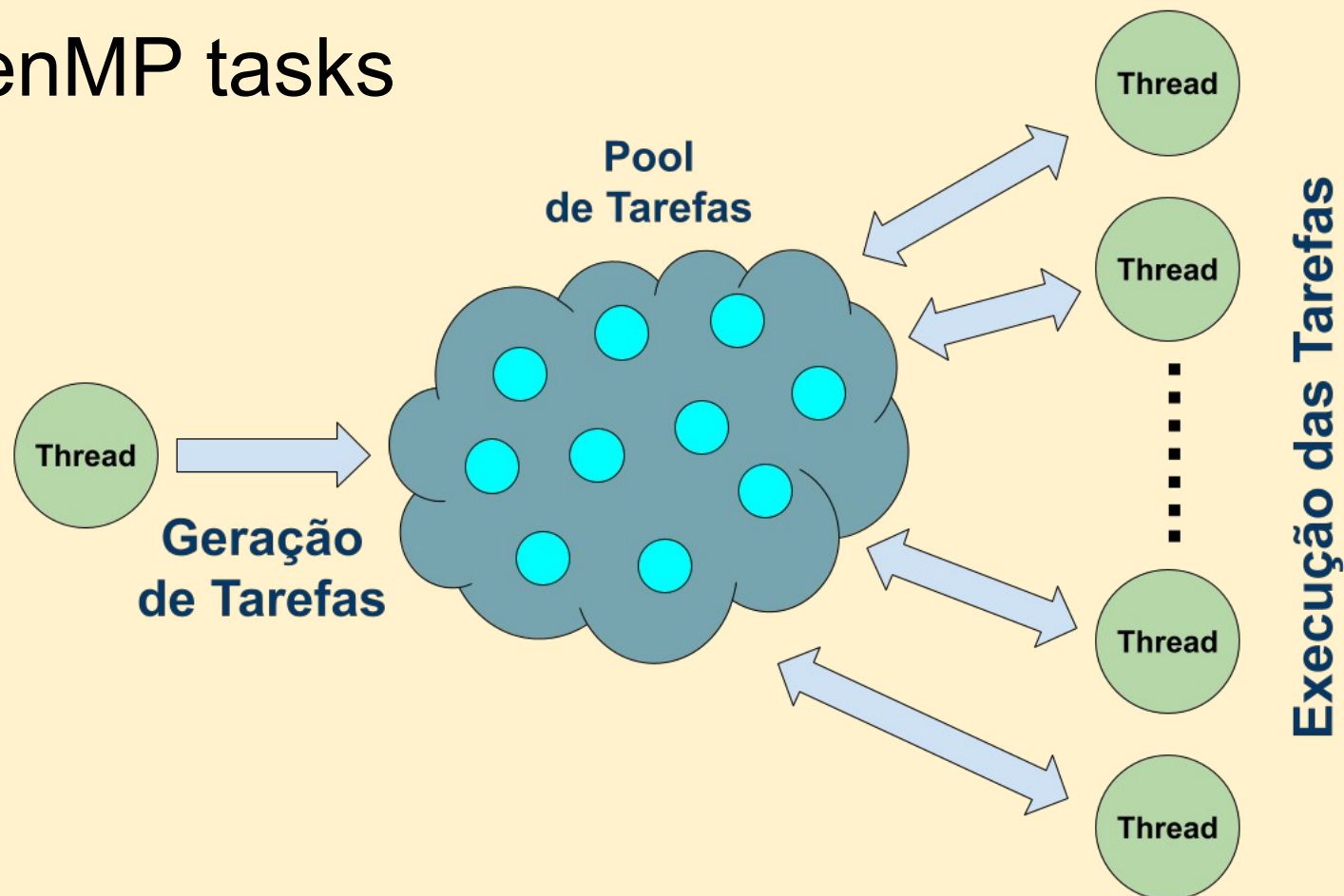
OpenMP tasks

- As tarefas são unidades independentes de trabalho.
- As tarefas são compostas por:
 - código a ser executado
 - ambiente de dados
 - variáveis de controle internas
- As *threads* realizam o trabalho determinado por cada tarefa.
- As tarefas são criadas:
 - ao alcançar uma região paralela → **tarefas implícitas** são criadas (uma por *thread*)
 - ao encontrar uma construção de **task** → uma **tarefa explícita** é criada
 - ao encontrar uma construção de **taskloop** → **tarefas explícitas** para cada bloco (*chunk*) são criadas
 - ao encontrar uma construção **target** → uma **tarefa target** é criada

OpenMP tasks

- O ambiente de execução (*runtime system*) decide quando as tarefas são executadas:
 - Elas podem ser adiadas ou executadas imediatamente.
 - O término de uma tarefa pode ser forçado pela sincronização de tarefas.
- Se a execução da tarefa for adiada, a tarefa será colocada em um *pool* de tarefas.
- Uma *thread* que executa uma tarefa pode diferir da *thread* que a encontrou originalmente.

OpenMP tasks



```
#pragma omp task [clause]
{
    ...
}
```

OpenMP tasks

- OpenMP define os seguintes pontos para o **escalonamento** de tarefas:
 - O ponto de encontro de uma construção **task**;
 - O ponto de encontro de uma construção **taskwait**;
 - O ponto de encontro de uma construção **taskyield**;
 - O ponto de encontro de uma barreira (**barrier**) implícita ou explícita;
 - O ponto de conclusão de uma **task**.
- Todas as tarefas explícitas geradas dentro de uma região paralela têm a garantia de serem completadas na saída da próxima barreira implícita ou explícita dentro da região paralela.

OpenMP tasks

- `#pragma omp task [cláusula[,cláusula]*]`

- **if (expression)**
- **final (expression)** - Quando a expressão final é avaliada como verdadeira, a tarefa não terá descendentes.
- **mergeable** – Uma tarefa mesclada é uma tarefa cujo ambiente de dados é o mesmo que a região da tarefa geradora.

Estratégia de encerramento

- **untied** – Se a tarefa estiver vinculada, é garantido que uma mesma *thread* executa todas as partes da tarefa, mesmo que a execução da tarefa tenha sido temporariamente suspensa.
- **priority (value)** – Um valor numérico não negativo, que recomenda que uma tarefa com alta prioridade seja executada antes de uma tarefa com menor prioridade.

Escalonamento das tasks

- **default (shared | firstprivate | none)**
- **private (list)**
- **firstprivate (list)**
- **shared (list)**

Escopo das variáveis

- **depend (list)** – Veja exemplo

Sincronização

OpenMP tasks

- Construtor de tarefa (*task construct*) – Uma diretiva `task` mais um bloco estruturado.
- Região de tarefa (*task region*) – um sequência dinâmica de instruções produzidas pela execução de uma tarefa por uma *thread*.

```
#pragma omp parallel num_threads (8)
{
    #pragma omp task
    foo() ;
    #pragma omp barrier
    #pragma omp single
    {
        #pragma omp task
        bar() ;
    }
}
```

Várias tarefas “foo” são criadas aqui.

Todas as tarefas “foo” terminam garantidamente aqui.

Uma tarefa “bar” criada aqui.

A tarefa “bar” termina garantidamente aqui.

Exemplo #1

```
#include <stdlib.h>
#include <stdio.h>

int main( int argc, char * argv[]) {
#pragma omp parallel
{
    printf ("A ");
    printf ("race ");
    printf ("car ");
} // Fim da Região Paralela

printf ("\n");
return(0);
}
```

As threads são executadas em paralelo. Não há garantia da ordem de execução.

```
$ gcc -fopenmp hello.c
$ export OMP_NUM_THREADS=2
$ ./a.out
A race car A race car
$ ./a.out
A race A car race car
```


Exemplo #2

```
#include <stdlib.h>
#include <stdio.h>

int main( int argc, char * argv[]) {
#pragma omp parallel
#pragma omp single
{
    printf ("A ");
    #pragma omp task
    printf ("race ");
    #pragma omp task
    printf ("car ");
} // Fim da Região Paralela

printf ("\n");
return(0);
}
```

As tarefas podem ser executadas em qualquer ordem.

```
$ gcc -fopenmp hello.c
$ export OMP_NUM_THREADS=2
$ ./a.out
A race car
$ ./a.out
A race car
$ ./a.out
A car race
```

Exemplo #3

```
#include <stdlib.h>
#include <stdio.h>

int main( int argc, char * argv[]) {
#pragma omp parallel
#pragma omp single
{
    printf ("A ");
#pragma omp task
    printf ("race ");
#pragma omp task
    printf ("car ");
    printf("is fun to watch ");
} // Fim da Região Paralela

printf ("\n");
return(0);
}
```

As tarefas são executadas no ponto de execução das tasks.

```
$ gcc -fopenmp hello.c
$ export OMP_NUM_THREADS=2
$ ./a.out
A is fun to watch race car
$ ./a.out
A is fun to watch race car
$ ./a.out
A is fun to watch car race
```

Exemplo #4

```
#include <stdlib.h>
#include <stdio.h>

int main( int argc, char * argv[]) {
#pragma omp parallel
#pragma omp single
{
    printf ("A ");
#pragma omp task
    printf ("race ");
#pragma omp task
    printf ("car ");
    #pragma omp taskwait
    printf("is fun to watch ");
} // Fim da Região Paralela

printf ("\n");
return(0);
}
```

As tarefas são executadas primeiro agora.

```
$ gcc -fopenmp hello.c
$ export OMP_NUM_THREADS=2
$ ./a.out
A car race is fun to watch
$ ./a.out
A car race is fun to watch
$ ./a.out
A race car is fun to watch
```

Exemplo Fibonacci

- Programa para cálculo dos números de Fibonacci
- Os números de Fibonacci são números na seguintes sequência inteira.

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144,

- Em termos matemáticos, a sequência F_n dos números de Fibonacci é definida pela seguinte relação de recorrência:

$$F_n = F_{n-1} + F_{n-2}$$

- Com os seguintes valores iniciais

$$F_0 = 0 \text{ e } F_1 = 1.$$

Exemplo Fibonacci

Sequential code

```
int main(  
    int argc,  
    char* argv[])  
{  
  
    fib(N);  
  
}
```

```
int fib(  
    int n  
) {  
    if (n < 2) return n;  
    int x = fib(n - 1);  
    int y = fib(n - 2);  
    return x + y;  
}
```

Escopo das variáveis

```
int fib (int n) {  
    int x,y;  
    if ( n < 2 ) return n;  
    #pragma omp task  
    x = fib(n-1);  
    #pragma omp task  
    y = fib(n-2);  
    #pragma omp taskwait  
    return x+y;  
}
```

n, x e y são privados em ambas tarefas

As variáveis privadas de um tarefa são indefinidas fora da tarefa.

Escopo das variáveis

```
int fib (int n) {  
    int x,y;  
    if (n < 2) return n;  
    #pragma omp task shared (x)  
    x = fib(n-1);  
    #pragma omp task shared (y)  
    y = fib(n-2);  
    #pragma omp taskwait  
    return x+y  
}
```

n é privado em ambas tarefas

***Declarar x e y
compartilhados é uma boa
solução***

**Solução correta do ponto de vista funcional,
porém com baixo desempenho, porque a
granularidade das tarefas é muito pequena. O
desempenho pode ser melhorado utilizando
estratégias de corte (cláusulas if ou final).**

Exemplo Fibonacci

Parallel code

```
int main(  
    int argc,  
    char* argv[])  
{  
  
    #pragma omp parallel  
    {  
        #pragma omp single  
        {  
            fib(N);  
        }  
    }  
}
```

```
int fib(  
    int n  
) {  
    if (n < 2) return n;  
    int x, y;  
    #pragma omp task shared(x)  
    {  
        x = fib(n - 1);  
    }  
    #pragma omp task shared(y)  
    {  
        y = fib(n - 2);  
    }  
    #pragma omp taskwait  
    return x + y;  
}
```


Escopo das Variáveis

- Escopo de dados das cláusulas:

`shared(list)`, `private(list)`, `firstprivate(list)`

- Variáveis estáticas e globais são **compartilhadas**.
- Variáveis de armazenamento automático (locais) são **privadas**.
- Variáveis de tarefas órfãs são **firstprivate** por padrão.
- Variáveis de tarefas com pais herdam os atributos de compartilhamento da tarefa pai.
- Variáveis são **firstprivate** a menos que compartilhadas no contexto em que estão colocadas.

Escopo das Variáveis

Variáveis com duração de armazenamento automática declaradas dentro da construção task são privadas.

```
#pragma omp task
{
    int x = MN;
    // Escopo de x: privado
}
```

Variáveis com duração de armazenamento estática declaradas dentro da construção task são compartilhadas.

```
#pragma omp task
{
    static int y
    // Escopo de y: compartilhado
}
```

Objetos de armazenamento automático são compartilhados (malloc, new,...)

```
int *p;
p = malloc(sizeof(float)*SIZE);
#pragma omp task
{
    // *p: shared
}
```

Escopo das Variáveis

```
int a = 1;
void foo() {
    int b = 2, c = 3;
    #pragma omp parallel private(b) {
        int d = 4;
        #pragma omp task
        {
            int f = 5;
            // Escopo de a:
            // Escopo de b:
            // Escopo de c:
            // Escopo de d:
            // Escopo de f:
        }
    }
}
```

Escopo das Variáveis

```
int a = 1;
void foo() {
    int b = 2, c = 3;
    #pragma omp parallel private(b)
    {
        int d = 4;
        #pragma omp task
        {
            int f = 5;
            // Escopo de a: shared
            // Escopo de b: private
            // Escopo de c: shared
            // Escopo de d: firstprivate
            // Escopo de f: private
        }
    }
}
```

Escopo das Variáveis – na dúvida use default (none)

```
int a = 1;
void foo() {
    int b = 2, c = 3;
    #pragma omp parallel private(b) {
        int d = 4;
        #pragma omp task
        {
            int f = 5;
            // Escopo de a: shared          valor de a: 1
            // Escopo de b: private valor de b: 0/indefinido
            // Escopo de c: shared          valor de c: 3
            // Escopo de d: firstprivate   valor de d: 4
            // Escopo de f: private        valor de f: 5
        }
    }
}
```

Exemplo Quicksort

- Um algoritmo frequentemente utilizado para ordenação é o quicksort.
- Utiliza uma estratégia de dividir para conquistar com os seguintes passos:
 - Dividir o vetor com um pivô, de tal modo que:
 - Todos os elementos à esquerda são menores
 - Todos os elementos à direita são maiores ou iguais
 - Repetir o procedimento para a parte direita e esquerda até completar

Exemplo Quicksort

- Um algoritmo recursivo sequencial

```
1 void Quicksort(int64_t *a, int64_t lo, int64_t hi)
2 {
3     if ( lo < hi ) {
4         int64_t p = partitionArray(a, lo, hi);
5
6         (void) Quicksort(a, lo, p - 1); // Left branch
7
8         (void) Quicksort(a, p + 1, hi); // Right branch
9     }
10 }
```

Exemplo Quicksort

```
1 #pragma omp parallel default(none) shared(a,nelements)
2 {
3     #pragma omp single nowait
4     { (void) Quicksort(a, 0, nelements-1); }
5 } // End of parallel region
```

```
1 void Quicksort(int64_t *a, int64_t lo, int64_t hi)
2 {
3     if ( lo < hi ) {
4         int64_t p = partitionArray(a, lo, hi);
5
6         #pragma omp task default(none) firstprivate(a,lo,p)
7         {(void) Quicksort(a, lo, p - 1);} // Left branch
8
9         #pragma omp task default(none) firstprivate(a,hi,p)
10        {(void) Quicksort(a, p + 1, hi);} // Right branch
11    }
12 }
```


Exemplo Quicksort

- Quando o tamanho do vetor for muito pequeno, é melhor utilizar o algoritmo sequencial.
- Isso pode ser realizado com a cláusula `if`.
- O modelo tradicional de *thread* pode ser considerado desatualizado
- O uso de tasks é mais flexível e alivia o programador de pensar no mapeamento da execução das tarefas para as *threads*.

Exemplo Lista Encadeada

```
my_pointer = listhead;
```

```
.....
```

```
while(my_pointer) {  
    (void) do_independent_work (my_pointer)  
    my_pointer = my_pointer->next ;  
} // fim do laço while
```

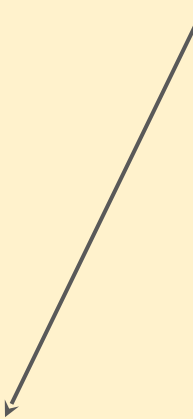
```
.....
```

Difícil de realizar sem utilizar tasks: primeiro contar o número de iterações e depois converter o while para um laço for.

Exemplo Lista Encadeada

```
my_pointer = listhead;
#pragma omp parallel
{
    #pragma omp single
    {
        while(my_pointer) {
            #pragma omp task firstprivate(my_pointer) {
                (void) do_independent_work (my_pointer);
            }
            my_pointer = my_pointer->next ;
        }
    } // Fim do single
} // Fim da região paralela
```

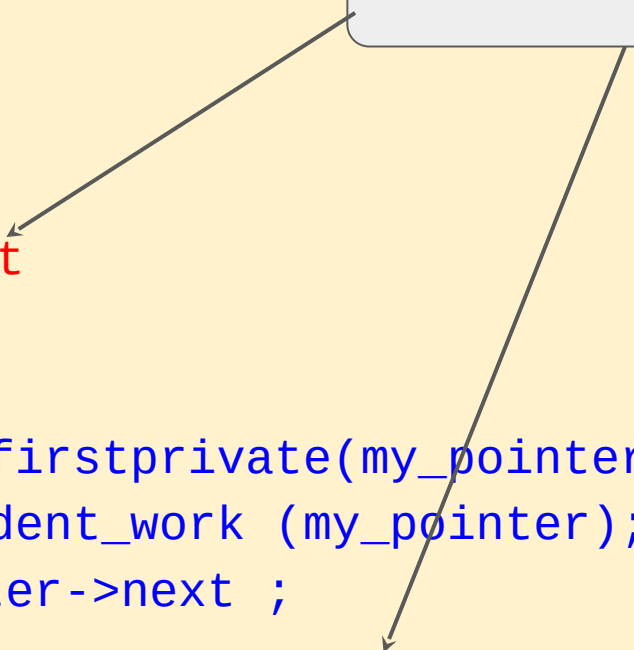
A tarefa OpenMP Task é especificada aqui (executadas em paralelo)



Exemplo Lista Encadeada

Pode eliminar uma barreira

```
my_pointer = listhead;
#pragma omp parallel
{
    #pragma omp single nowait
    {
        while(my_pointer) {
            #pragma omp task firstprivate(my_pointer)
            (void) do_independent_work (my_pointer);
            my_pointer = my_pointer->next ;
        }
    } // Fim do single - sem barreira implícita (nowait)
} // Fim da região paralela - barreira implícita
```



Dependência de Tarefas

- Uma tarefa não pode ser executada até que todas as suas tarefas predecessoras sejam concluídas.
- Se uma tarefa definir uma dependência **in** sobre um item de lista
 - a tarefa dependerá de todas as tarefas irmãs geradas anteriormente que referenciam esse item de lista em uma dependência **out** ou **inout**.
- Se uma tarefa definir uma dependência **out** ou **inout** sobre um item de lista
 - a tarefa dependerá de todas as tarefas irmãs geradas anteriormente que referenciam esse item de lista em uma dependência **in**, **out** ou **inout**.
- Em outras palavras, as tarefas OpenMP podem depender umas das outras com base nos dados que elas acessam.

Dependência de Tarefas

- Uma dependência **in** significa que a tarefa precisa dos dados produzidos por outra tarefa para ser executada.
- Uma dependência **out** significa que a tarefa produz dados que serão usados por outra tarefa.
- Uma dependência **inout** significa que a tarefa produz dados que serão usados por outra tarefa e que serão modificados pela outra tarefa.
- As dependências entre tarefas são especificadas usando a cláusula **depend** da diretiva **task**. A cláusula **depend** pode ser usada para especificar dependências **in**, **out** e **inout**.

Dependência de Tarefas

```
#pragma omp task depend(dependency-type: list)
```

- ▶ The *task dependence* is fulfilled when the predecessor task has completed
 - ▶ in dependency-type: the generated task will be a dependent task of all previously generated sibling tasks that reference at least one of the list items in an out or inout clause.
 - ▶ out and inout dependency-type: the generated task will be a dependent task of all previously generated sibling tasks that reference at least one of the list items in an in, out, or inout clause.

```
#pragma omp task  
  x = f();  
#pragma omp task  
  y = g(x);
```

```
#pragma omp task depend(out:x)  
  x = f();  
#pragma omp task depend(in:x)  
  y = g(x);
```

Dependência de Tarefas

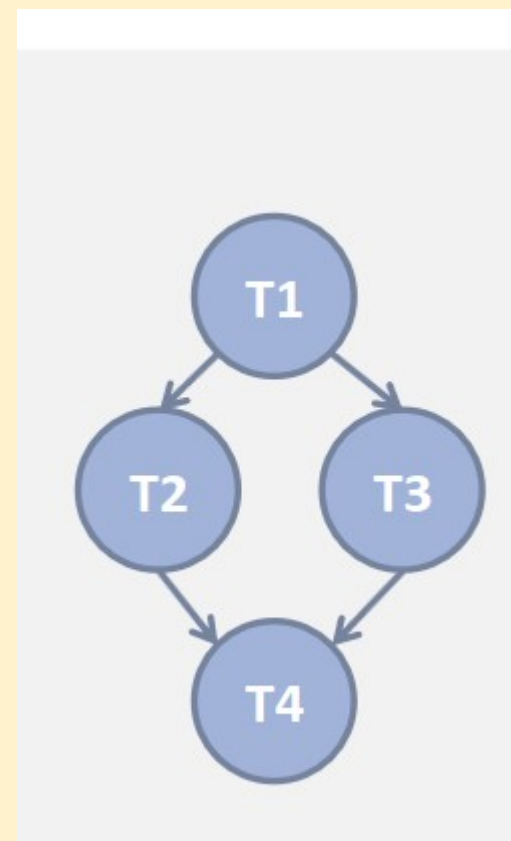
```
int x = 0;
#pragma omp parallel
#pragma omp single

{
#pragma omp task depend(inout: x) //T1
{ ... }

#pragma omp task depend(in: x) //T2
{ ... }

#pragma omp task depend(in: x) //T3
{ ... }

#pragma omp task depend(inout: x) //T4
{ ... }
}
```



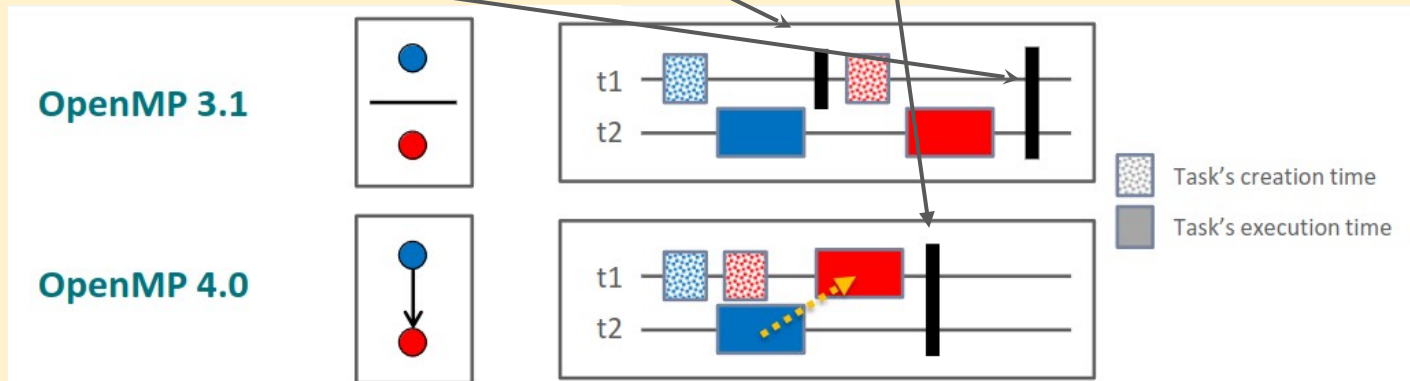
Dependência de Tarefas

OpenMP 3.1

```
int x = 0;
#pragma omp parallel
#pragma omp single
{
    #pragma omp task
    printf("%d\n", x);
    #pragma omp taskwait
    #pragma omp task
    x++;
}
```

OpenMP 4.5

```
int x = 0;
#pragma omp parallel
#pragma omp single
{
    #pragma omp task depend(in: x)
    printf("%d\n", x);
    #pragma omp task depend(inout: x)
    x++;
}
```



Dependência de Tarefas

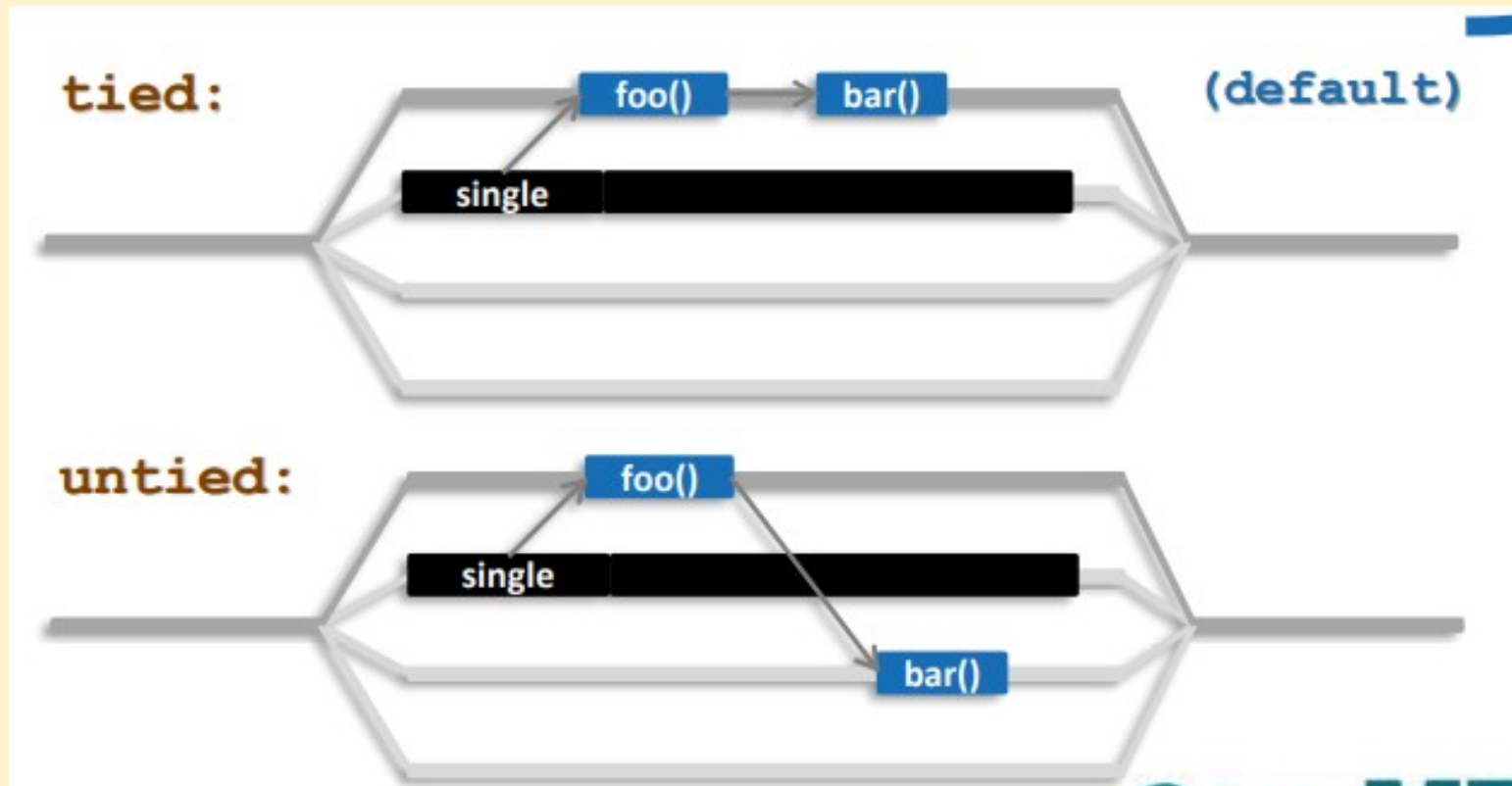
```
void process_in_parallel( ) {  
    # pragma omp parallel  
    # pragma omp single  
    {  
        int x = 1;  
        ...  
  
        for (int i = 0; i < T; ++i) {  
            # pragma omp task shared (x, ...) depend (out: x)  
            // T1  preprocessa_algun_dado(...);  
  
            # pragma omp task shared (x, ...) depend (in: x)  
            // T2  faz_algumacoisa_com_dado (...);  
  
            # pragma omp task shared (x, ...) depend (in: x)  
            // T3  faz_algumacoisa_independente_com_dado (...);  
        }  
    } // fim do single e parallel  
}
```

Cláusula untied

```
#pragma omp task untied {structured-block}
```

- As tarefas são vinculadas por padrão (quando nenhuma cláusula **untied** está presente)
 - Tarefas vinculadas são sempre executadas pela mesma *thread* (não necessariamente a que a criou)
 - Tarefas vinculadas podem apresentar problemas de desempenho
- Os programadores podem especificar que as tarefas sejam desvinculadas (agendadas com flexibilidade)
 - Podem potencialmente ser executadas por qualquer thread (da equipe)
 - Não se harmoniza bem com recursos baseados em *threads*: *thread-id*, *threadprivate*, regiões critical ...
 - Isso dá ao sistema de execução mais flexibilidade para agendar as tarefas
 - Mas a maioria das implementações OpenMP não "respeita" as tarefas desvinculadas.

Cláusula untied



Diretivas barrier e taskwait

- **Diretiva barrier**

- Todas as tarefas criadas por qualquer *thread* da equipe atual garantidamente são completadas na saída da diretiva **barrier**
- E em todas as outras barreiras implícitas em **parallel**, **sections**, **for**, **single**, etc.

pragma omp barrier

- **Diretiva taskwait**

- A tarefa que encontra esta diretiva é suspensa até que as tarefas filhas sejam completadas.
- Aplica-se apenas aos filhos diretos, não aos descendentes!
- Inclui um ponto implícito de agendamento de tarefas (TSP).

pragma omp taskwait

Exemplo Cholesky

```
void cholesky(int ts, int nt, double* a[nt][nt]) {  
    for (int k = 0; k < nt; k++) {  
        potrf(a[k][k], ts, ts);  
        // Sistemas Triangulares  
        for (int i = k + 1; i < nt; i++) {  
            #pragma omp task  
            trsm(a[k][k], a[k][i], ts, ts);  
        }  
        #pragma omp taskwait  
        // Atualiza matrix trailingx  
        for (int i = k + 1; i < nt; i++) {  
            for (int j = k + 1; j < i; j++) {  
                #pragma omp task  
                dgemm(a[k][i], a[k][j], a[j][i], ts  
            }  
            #pragma omp task  
            syrk(a[k][i], a[i][i], ts, ts);  
        }  
        #pragma omp taskwait  
    }  
}
```

Padrões de sincronização complexos podem ser resolvidos dividindo fases computacionais e usando construções como "taskwait" ou "taskgroup." Isso, no entanto, exige uma análise de código complexa. A melhoria da programabilidade pode ser alcançada ao considerar as dependências do OpenMP, o que também aprimora a composição de código.

Diretiva taskgroup

- **Diretiva taskgroup**

pragma omp taskgroup [cláusula]

- Especifica a espera pelo término das tarefas filhas e dos seus descendentes.
- Sincronização mais profunda que **taskwait**
- Pode ser restrita a um subconjunto de tarefas (ao contrário da barreira)
- Pode ser usada para o cancelamento da tarefa
- onde cláusula pode ser: **task_education(reduction-identifier: list-items)** ou **allocate**

Diretivas taskwait vs taskgroup

center@#01%101

Taskwait

```
int main()
{
    #pragma omp parallel
    #pragma omp single
    {
        #pragma omp task
        {
            #pragma omp critical
            printf ("Task_1\n");

            #pragma omp task
            {
                sleep(1);
                #pragma omp critical
                printf ("Task_2\n");
            }

            #pragma omp taskwait

            #pragma omp task
            {
                #pragma omp critical
                printf ("Task_3\n");
            }
        }
    }
}
```

Taskgroup

```
int main()
{
    #pragma omp parallel
    #pragma omp single
    {
        #pragma omp taskgroup
        {
            #pragma omp task
            {
                #pragma omp critical
                printf ("Task_1\n");

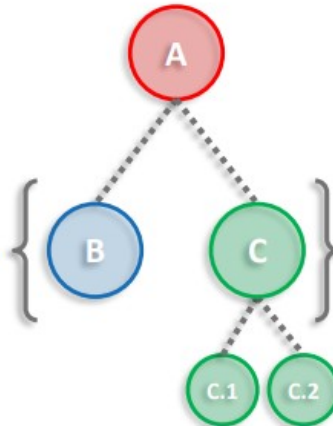
                #pragma omp task
                {
                    sleep(1);
                    #pragma omp critical
                    printf ("Task_2\n");
                }
            }
        } /* end of taskgroup */

        #pragma omp task
        {
            #pragma omp critical
            printf ("Task_3\n");
        }
    }
}
```


Diretivas taskwait vs taskgroup

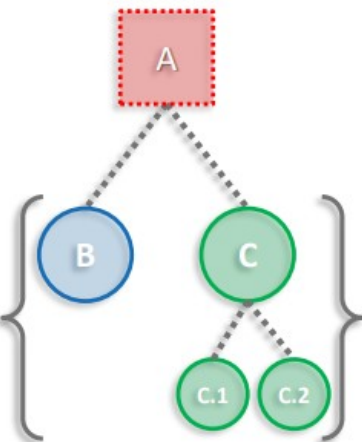
```
#pragma omp parallel
#pragma omp single
{
    #pragma omp task :A
    {
        #pragma omp task :B
        { ... }
        #pragma omp task :C
        { ... #C.1; #C.2; ... }
        #pragma omp taskwait
    }
} // implicit barrier will wait for C.x
```

wait for...



```
#pragma omp parallel
#pragma omp single
{
    #pragma omp taskgroup :A
    {
        #pragma omp task :B
        { ... }
        #pragma omp task :C
        { ... #C.1; #C.2; ... }
    } // end of taskgroup
}
```

wait for...



Diretiva taskyield

- Pontos de agendamento de tarefas (e na diretiva `taskyield`)
 - → Tarefas podem ser suspensas/reiniciadas nos Pontos de Agendamento de Tarefas (TSPs) → com algumas restrições adicionais para evitar problemas de bloqueio
 - Pontos de agendamento implícitos (criação, sincronização, ...)
 - Ponto de agendamento explícito: a diretiva `taskyield`

```
#pragma omp taskyield
```

```
#pragma omp parallel
#pragma omp single
{
    #pragma omp task
    {
        foo();
        #pragma omp taskyield
        bar()
    }
}
```

Diretiva taskloop

- Paraleliza um laço utilizando OpenMP tasks
- Divide o laço em partes
- Cria uma task para cada parte do laço
- Sintaxe (C/C++)

```
#pragma omp taskloop [simd] [clause[.,] clause],...]  
for-loops
```

Diretiva taskloop

- A diretiva **taskloop** pode utilizar as seguintes cláusulas:
 - **grainsize**(grain-size)
 - Os blocos têm entre **grain-size** e $2 * \text{grain-size}$ iterações de laço
 - **num_tasks**(num-tasks)
 - Cria **num-tasks** tarefas para executar as iterações do laço
 - Se nenhuma das cláusulas anteriores estiver presente, o número de blocos e o número de iterações por bloco são definidos pela implementação.
 - **nogroup**
 - A construção **taskloop** cria uma região **taskgroup** implícita. Se a cláusula **nogroup** estiver presente, nenhuma região de **taskgroup** implícita será criada.

Diretiva taskloop

- A diretiva **taskloop** pode utilizar as seguintes cláusulas:
 - shared, private, firstprivate, lastprivate
 - default (shared, none)
 - collapse(n)
 - final (expressao-escalar)
 - if ([taskloop :] expressao-escalar)
 - untied, mergeable
 - depend (tipo-de-dependencia : lista)
 - priority (valor-da-prioridade)

Diretiva taskloop

- ▶ Parallelizes loop by creating tasks for one or more iterations of the loop.
- ▶ Cut loops into chunks and create a task for each loop chunk.
- ▶ Inherits clauses from worksharing and task construct.
- ▶ Provides better load balancing in some cases.
- ▶ Attitional clauses:
 - ▶ `grainsize(grain-size)`: chunks have at least grain-size and max $2 * \text{grain-size}$ loop iterations,
 - ▶ `num_tasks(num-tasks)`: create num-tasks tasks for iterations of the loop.

```
#pragma omp parallel shared(a, b, c)
{
    #pragma omp single
    {
        #pragma omp task
        long_running_comp();                // can execute concurrently
        #pragma omp taskloop grainsize(1000)
        for (int i = 0; i < 1000000; ++i) {  // can execute concurrently
            c[i] = a[i] + b[i];
        }
    }
}
```

Diretiva taskloop simd

- Divide um laço em blocos, cria uma tarefa para cada bloco do laço.
- Cada tarefa gerada aplicará (internamente) SIMD a cada bloco do loop.
- Admite quaisquer das cláusulas permitidas pelas diretivas `taskloop` ou `simd`.

```
#pragma omp taskloop simd [clause[[,] clause]...]  
    {laço-for-estruturado}
```

Referências

https://blog.rwth-aachen.de/itc-events/files/2021/02/11-openmp-CT-tasking_dependencies.pdf

<https://hpc2n.github.io/Task-based-parallelism/branch/master/task-basics-1/>

<https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>

<https://openmp.org/wp-content/uploads/sc13.tasking.ruud.pdf>

<https://www.openmp.org/wp-content/uploads/OpenMP-UMT-Tasking-1.pdf>

[https://hpc-wiki.info/hpc/OpenMP in Small Bites/Tasking and Data Scoping](https://hpc-wiki.info/hpc/OpenMP_in_Small_Bites/Tasking_and_Data_Scoping)